

VaaniAI – Multilingual AI Healthcare Receptionist

Version: 1.2 (Final Draft) Date: 2026-05-20 Author: [Your Name]

1 Executive Summary

VaaniAI is a voice-first, multilingual (English + Hindi + Hinglish) AI receptionist that lets patients of small clinics book appointments and get basic doctor-information entirely by speaking.

The MVP is deliberately scoped to a single-clinic demo that can be run in an interview, not a production-grade hospital platform.

The system showcases three core capabilities:

Capability	How it’s demonstrated	Why it matters to interviewers
Voice-to-Booking	Speak a request → real-time STT → LLM decides next step → TTS → audible answer → booking created in PostgreSQL	Shows end-to-end AI-driven workflow (audio-in / audio-out)
Multilingual handling	Same flow works for pure English, pure Hindi, or Hinglish (code-mixed) utterances	Demonstrates language model usage & language-agnostic design
Live dashboard	Streamlit UI instantly reflects calls, transcripts, bookings, language distribution	Gives a quantifiable “admin view” that you can walk through while narrating architecture

2 Product Vision & Value Proposition

Vision: “Every small clinic can answer its phone like a 24 × 7 receptionist without hiring extra staff, while patients can speak in the language they’re most comfortable with.”

Value:

- For patients – instant, natural-language access to appointments without navigating forms.
 - For clinics – reduced call-handling load, a log of interactions, and a simple way to measure demand per doctor/specialty.
-

3 Problem Statement

Small clinics receive a high volume of repetitive calls (“Which doctor is free tomorrow?” “Can I see a dermatologist?”). Staff time spent on these queries limits capacity and frustrates patients, especially those who prefer speaking Hindi or Hinglish. Existing solutions are either manual (human receptionist) or text-only, requiring patients to type in a language they may not be comfortable with.

4 Personas & Use-Cases

Persona	Primary Goals	Core Use-Case
A. Patient – “Rahul”	Book a dermatologist quickly, speak Hindi.	“Mujhe skin doctor chahiye, kal 10 baje.”
B. Patient – “Emily”	Verify doctor availability, speak English.	“Do you have any general physicians free tomorrow?”
C. Clinic Admin / Interviewer	Observe how many calls/bookings happen, check transcript quality.	Open Streamlit dashboard → see call #, language, booking.

5 Success Metrics (OKRs)

Objective	Key Result (target)
Product viability	✔ End-to-end voice booking completed ≤ 2 minutes in demo.
Multilingual coverage	✔ ≥ 90 % of test utterances in English/Hindi/Hinglish transcribed with ≤ 15 % word-error-rate.
Data capture	✔ 100 % of user turns stored as transcript rows.
Dashboard visibility	✔ New booking appears on Streamlit within 5 seconds of confirmation.
Reliability for demo	✔ No unhandled exceptions across 5 consecutive demo runs.
Explainability	✔ Ability to describe the entire stack in ≤ 2 minutes.

6 Scope – What’s In/Out

In-Scope	Out-Of-Scope (MVP)
• Real-time speech → text (Faster-Whisper). • LLM response (Gemini Flash). • Text-to-speech (XTTS).	• Kubernetes, Redis, or any heavy orchestration. • Human escalation or live-agent handoff. • Insurance, billing, or prescription generation. • Calendar sync with Google/Outlook.

In-Scope

- PostgreSQL storage (users, doctors, appointments, transcripts, call logs).
- Simple deterministic state-machine for booking.
- Multilingual phrasing (English/Hindi/Hinglish).
- Streamlit admin dashboard.
- Vapi webhook for phone-call demo (optional – local mic works too).

Out-Of-Scope (MVP)

- Multi-tenant clinic support.
- SMS/WhatsApp reminders.
- Full HIPAA-level audit / encryption (we'll note security basics but not implement production-grade).

7 Functional Requirements

#	Feature	Description	Acceptance Criteria
1	Audio capture	Accept audio via local mic (Web UI) or Vapi webhook.	- Audio file/stream reaches /transcribe. - Call ID is generated and logged. - Returns JSON {text, language} within 2 s for ≤ 10 s clip.
2	Speech-to-Text	Faster-Whisper model (large-v2) transcribes English/Hindi/Hinglish.	- Language detection is at least correct for 90 % of test samples.
3	Conversation Engine	Stateless FastAPI endpoint /chat that receives user text, current call_id, and runs: (a) intent detection, (b) deterministic state-machine transition, (c) LLM response generation.	- Returns reply, next_state, optional action (e.g., show_slots). - No “hallucinated” medical advice; any health-question triggers safe-fallback.
4	Intent Detection	Light rule-based + small prompt to Gemini Flash for intents: book_appointment, doctor_inquiry, language_switch, repeat_last_doctor, unknown.	- Unit tests cover at least 15 sample utterances per intent.
5	Doctor & Slot Lookup	SQL queries on doctors and slots tables.	- Returns sorted list of available slots (future dates only). - No double-booking (unique constraint on (doctor_id, slot_time)).
6	Appointment Booking	After confirming all required fields, create appointments row with status confirmed.	- Booking ID returned. - Slot becomes unavailable for subsequent calls.

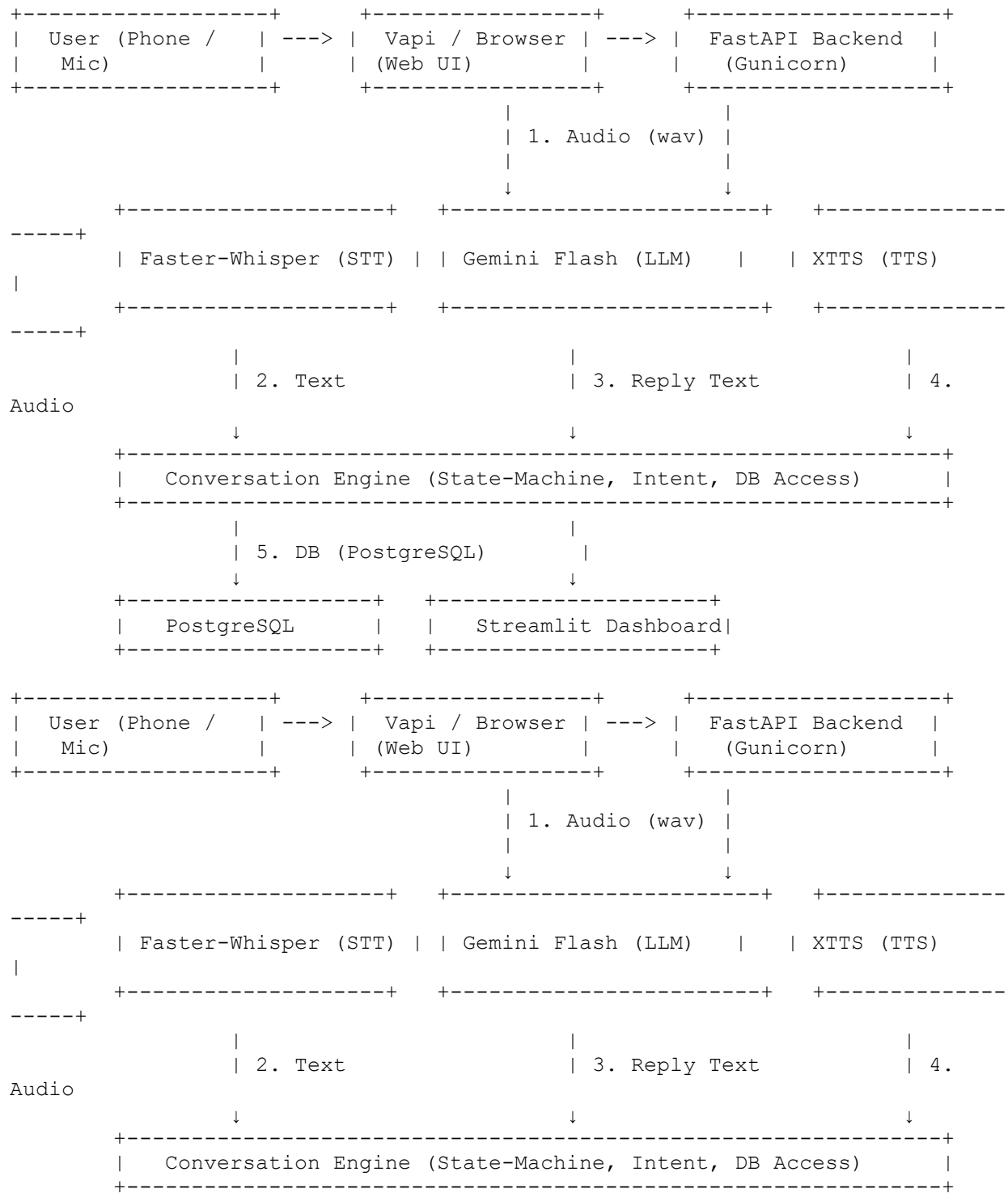
#	Feature	Description	Acceptance Criteria
7	Memory (lightweight profile)	On each call, fetch/create <code>users</code> row by <code>phone</code> (or session ID). Store <code>preferred_language</code> , <code>preferred_doctor_id</code> , <code>last_appointment_id</code> .	- Returning callers can invoke “same doctor” flow.
8	Transcript Persistence	Every turn (user & assistant) is inserted into <code>transcripts</code> with <code>call_id</code> , <code>speaker</code> , <code>text</code> , <code>language</code> , <code>timestamp</code> .	- Dashboard query <code>SELECT * FROM transcripts WHERE call_id = ? ORDER BY timestamp</code> returns full conversation.
9	Text-to-Speech	XTTS model (<code>xtts_v2</code>) receives LLM reply text + language code → generates short WAV/MP3 (< 5 s).	- Audio file < 2 s to generate. - Playback works in web UI and Vapi response.
10	Dashboard (Streamlit)	4 sections: Metrics, Recent Bookings, Recent Transcripts, Doctor Demand.	- Refresh button updates data within 5 s. - Charts are generated with Plotly/Altair.
11	Vapi Integration (optional)	Webhook that receives inbound call audio, calls <code>/transcribe</code> , then <code>/chat</code> , then returns TTS audio.	- Simulated call “Hi, I need a skin doctor” results in audible confirmation and booking entry.
12	Error & Fallback Handling	Graceful fallbacks for (a) STT failure, (b) LLM timeout, (c) unknown intent.	- System replies “I’m sorry, I didn’t catch that. Could you repeat?” and logs <code>error_type</code> .

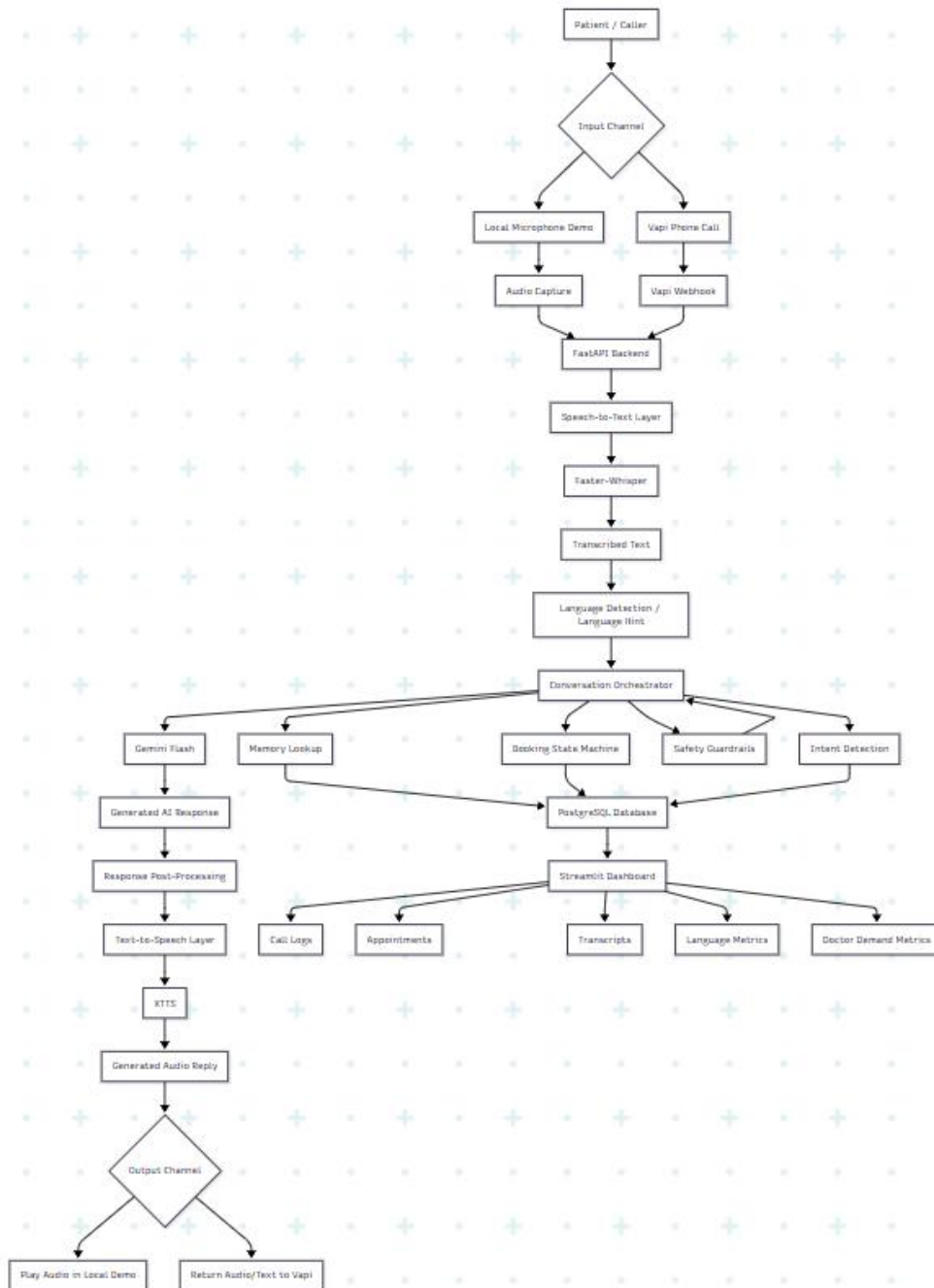
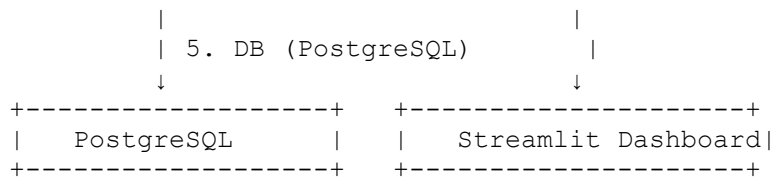
8 Non-Functional Requirements

Aspect	Requirement
Performance	End-to-end latency (audio → reply) ≤ 5 s for typical 8-second utterance.
Scalability	Designed for a single-clinic demo; can be scaled horizontally by adding more FastAPI workers (Gunicorn) if needed.
Reliability	All external calls (Whisper, Gemini, XTTS) wrapped with retries (max 3).
Security	- Use HTTPS (Render provides TLS). - Store DB credentials in environment variables. - Minimal PII: only <code>name</code> , <code>phone</code> ; no health data stored.
Observability	- Structured logs (JSON) for each request. - Simple health check <code>GET /health</code> . - Code base ≤ 2 k LOC.
Maintainability	- Typed with <code>pydantic</code> models. - Unit tests ≥ 80 % coverage.

Aspect	Requirement
Compliance	Not a HIPAA-compliant product (out-of-scope), but we'll document that no clinical information is persisted.

9 System Architecture (text diagram)





All components run on Render (free tier) or locally for interview demo.

10 Detailed Tech Stack

Layer	Tool	Reason
API	FastAPI + Uvicorn/Gunicorn	Rapid dev, async, auto-generated OpenAPI.
STT	Faster-Whisper (<code>large-v2</code>)	State-of-the-art multilingual transcription, runs on CPU (no GPU needed for short clips).
LLM	Gemini Flash (Vertex AI)	Low-latency, affordable, good multilingual reasoning.
TTS	XTTS (<code>xtts_v2</code>)	Open-source, supports Hindi/English grapheme-to-phoneme mapping.
DB	PostgreSQL (Render managed) + SQLAlchemy	Relational schema fits booking logic; ORM gives typed models.
Dashboard	Streamlit + Plotly	One-file UI, auto-reload, easy charts.
Phone Integration	Vapi (Twilio-like)	Webhook-first inbound call API, free trial for demo.
Containerization	Docker (single image)	Guarantees reproducible environment.
CI/CD	GitHub Actions (lint + test + Docker build)	Automatic sanity before each demo.
Version Control	Git (GitHub)	Standard for interview artifacts.

11 Data Model (SQLModel / ER diagram)

```
User
├── id (PK)
├── name
├── phone (unique)
├── language_preference
├── preferred_doctor_id (FK → Doctor.id)
└── created_at
```

```
Doctor
├── id (PK)
├── name
├── specialization
├── languages (JSON list)
├── active (bool)
└── created_at
```

```
Slot
├── id (PK)
├── doctor_id (FK → Doctor.id)
├── start_time (timestamp)
├── end_time (timestamp)
└── is_booked (bool, default false)
```

```
Appointment
├── id (PK)
```

- └ user_id (FK → User.id)
- └ doctor_id (FK → Doctor.id)
- └ slot_id (FK → Slot.id)
- └ status (enum: pending, confirmed, cancelled)
- └ created_at

CallLog

- └ id (PK)
- └ user_id (FK, nullable)
- └ call_sid (string from Vapi)
- └ started_at
- └ ended_at
- └ language_detected
- └ outcome (enum: booked, fallback, error)

Transcript

- └ id (PK)
- └ call_id (FK → CallLog.id)
- └ speaker (enum: user, assistant)
- └ text
- └ language
- └ timestamp

User

- └ id (PK)
- └ name
- └ phone (unique)
- └ language_preference
- └ preferred_doctor_id (FK → Doctor.id)
- └ created_at

Doctor

- └ id (PK)
- └ name
- └ specialization
- └ languages (JSON list)
- └ active (bool)
- └ created_at

Slot

- └ id (PK)
- └ doctor_id (FK → Doctor.id)
- └ start_time (timestamp)
- └ end_time (timestamp)
- └ is_booked (bool, default false)

Appointment

- └ id (PK)
- └ user_id (FK → User.id)
- └ doctor_id (FK → Doctor.id)
- └ slot_id (FK → Slot.id)
- └ status (enum: pending, confirmed, cancelled)
- └ created_at

CallLog

- └ id (PK)
- └ user_id (FK, nullable)
- └ call_sid (string from Vapi)
- └ started_at
- └ ended_at


```
└ language_detected
└ outcome (enum: booked, fallback, error)
```

```
Transcript
└ id (PK)
└ call_id (FK → CallLog.id)
└ speaker (enum: user, assistant)
└ text
└ language
└ timestamp
```

All timestamps are UTC. Unique constraint on (doctor_id, start_time) prevents double-booking.

12 API Specification (OpenAPI-style snippets)

12.1 POST /transcribe

```
json
{
  "summary": "Convert audio to text",
  "requestBody": {
    "content": {
      "multipart/form-data": {
        "schema": {
          "type": "object",
          "properties": {
            "audio": { "type": "string", "format": "binary" },
            "call_id": { "type": "string" }
          },
          "required": ["audio", "call_id"]
        }
      }
    }
  },
  "responses": {
    "200": {
      "description": "Transcription success",
      "content": {
        "application/json": {
          "example": {
            "text": "Mujhe skin doctor chahiye",
            "language": "hi"
          }
        }
      }
    }
  }
}

{
  "summary": "Convert audio to text",
  "requestBody": {
    "content": {
```

```

    "multipart/form-data": {
      "schema": {
        "type": "object",
        "properties": {
          "audio": { "type": "string", "format": "binary" },
          "call_id": { "type": "string" }
        },
        "required": ["audio", "call_id"]
      }
    }
  },
  "responses": {
    "200": {
      "description": "Transcription success",
      "content": {
        "application/json": {
          "example": {
            "text": "Mujhe skin doctor chahiye",
            "language": "hi"
          }
        }
      }
    }
  }
}

```

12.2 POST /chat

json

```

{
  "summary": "Advance conversation state",
  "requestBody": {
    "content": {
      "application/json": {
        "schema": {
          "type": "object",
          "properties": {
            "call_id": { "type": "string" },
            "user_id": { "type": "string" },
            "message": { "type": "string" },
            "language": { "type": "string" }
          },
          "required": ["call_id", "user_id", "message"]
        }
      }
    }
  },
  "responses": {
    "200": {
      "description": "LLM reply + next state",
      "content": {
        "application/json": {
          "example": {
            "reply": "Dr. Meera Sharma is available tomorrow at 10 AM and 4 PM.  
Which slot works?",
            "next_state": "collect_slot",
            "intent": "book_appointment",

```

```

        "action": "show_slots"
    }
}
}
}
}
{
    "summary": "Advance conversation state",
    "requestBody": {
        "content": {
            "application/json": {
                "schema": {
                    "type": "object",
                    "properties": {
                        "call_id": {"type": "string"},
                        "user_id": {"type": "string"},
                        "message": {"type": "string"},
                        "language": {"type": "string"}
                    },
                    "required": ["call_id", "user_id", "message"]
                }
            }
        }
    },
    "responses": {
        "200": {
            "description": "LLM reply + next state",
            "content": {
                "application/json": {
                    "example": {
                        "reply": "Dr. Meera Sharma is available tomorrow at 10 AM and 4 PM.  
Which slot works?",
                        "next_state": "collect_slot",
                        "intent": "book_appointment",
                        "action": "show_slots"
                    }
                }
            }
        }
    }
}
}
}
}

```

12.3 POST /book-appointment

```

json
{
    "summary": "Create a confirmed appointment",
    "requestBody": {
        "content": {
            "application/json": {
                "schema": {
                    "type": "object",
                    "properties": {
                        "user_id": {"type": "string"},
                        "doctor_id": {"type": "string"},
                        "slot_id": {"type": "string"}
                    },
                    "required": ["user_id", "doctor_id", "slot_id"]
                }
            }
        }
    }
}

```

```

        "required":["user_id","doctor_id","slot_id"]
    }
}
},
"responses":{
    "200":{
        "description":"Booking persisted",
        "content":{
            "application/json":{
                "example":{
                    "appointment_id":"c5a8b9f2-...",
                    "status":"confirmed"
                }
            }
        }
    }
}
}
{
    "summary":"Create a confirmed appointment",
    "requestBody":{
        "content":{
            "application/json":{
                "schema":{
                    "type":"object",
                    "properties":{
                        "user_id":{"type":"string"},
                        "doctor_id":{"type":"string"},
                        "slot_id":{"type":"string"}
                    },
                    "required":["user_id","doctor_id","slot_id"]
                }
            }
        }
    },
    "responses":{
        "200":{
            "description":"Booking persisted",
            "content":{
                "application/json":{
                    "example":{
                        "appointment_id":"c5a8b9f2-...",
                        "status":"confirmed"
                    }
                }
            }
        }
    }
}
}

```

(Endpoints for /doctors, /available-slots, /dashboard/metrics, /transcripts follow similar patterns.)

13 Conversation Flow & State Machine

13.1 Deterministic State Machine (core booking)

Current State	Input (Intent)	Transition	Action
idle	book_appointment	→ collect_specialization	Prompt: “Which doctor or specialty would you like?”
collect_specialization	specialization_selected	→ show_slots	Retrieve doctors + slots, present options.
show_slots	slot_selected	→ collect_name	Prompt “May I have your name?”
collect_name	name_provided	→ confirm_booking	Summarize: doctor, slot, name. Call /book-appointment; respond “Your appointment is confirmed.”
confirm_booking	affirmative	→ booked	“Okay, let’s start over.”
confirm_booking	negative	→ idle	Lookup previous doctor, present slots.
idle	repeat_last_doctor	→ show_slots (pre-filled)	Update user language_preference; re-prompt in new language.
idle	language_switch	→ idle	“I’m sorry, could you rephrase?”
any	unknown / fallback	→ idle	

Implementation tip: Store the current state in the `call_logs` table (`current_state` column). The `/chat` endpoint reads it, runs logic, updates it, and returns the next state.

13.2 Intent Mapping (rule-based + LLM)

Intent	Example Utterances	Detection Strategy
book_appointment	“I want to book”, “Appointment”, “Schedule”	Keyword list + prompt to LLM “Classify intent”.
doctor_inquiry	“Which doctor can treat a skin issue?”	Keyword-list + LLM fallback.
specialization_selected	“Dermatologist”, “Skin doctor”, “Skin”	Simple regex ↔ specialization dict.

Intent	Example Utterances	Detection Strategy
slot_selected	“Tomorrow 10 am”, “10 am”	Temporal parsing (<code>dateparser</code> lib) after extracting numbers.
name_provided	“My name is Rahul”, “Rahul”	Named-entity extraction via regex (capitalized word) + LLM.
affirmative	“Yes”, “Sure”, “Confirm”	Small set.
negative	“No”, “Cancel”, “Not now”	Small set.
language_switch	“Hindi mein baat karo”, “Speak in English”	Regex for language keywords.
repeat_last_doctor	“Same doctor as last time”, “Book with Dr. Meera again”	Phrase detection.
unknown	Anything else	Default fallback.

14 Dashboard Wireframe (text description)

1. Header – “VaaniAI Dashboard – Live Clinic Ops” (logo, last refresh timestamp).
2. Metrics Row (cards):
 - Total Calls (today)
 - Total Bookings (today)
 - Booking Conversion % (calls → bookings)
 - Top Language (pie-chart).
3. Recent Bookings Table – columns: `time`, `patient_name`, `doctor`, `slot`, `status`. Sort descending.
4. Recent Transcripts – expandable panel per Call ID; each entry shows alternating user / assistant bubbles with language badge.
5. Doctor Demand Bar Chart – X-axis: Doctor name, Y-axis: # of requests today.

All charts built with Plotly Express; data pulled via fast SQL queries (`@st.cache_data`).

15 Non-Functional & Ops Checklist

Item	Tool/Approach	Owner
Lint / Formatting	<code>ruff</code> , <code>black</code>	CI
Type-checking	<code>mypy</code>	CI
Tests	<code>pytest</code> (unit + API integration)	Dev
Docker	Multi-stage Dockerfile (builder + runtime)	DevOps
CI	GitHub Actions: lint → test → build → push Docker to GitHub Packages	DevOps

Item	Tool/Approach	Owner
Secrets	Render Environment Variables (<code>POSTGRES_URL</code> , <code>GEMINI_API_KEY</code> , <code>VAPI_TOKEN</code>)	Ops
Logging	<code>structlog</code> JSON -> stdout (captured by Render)	Dev
Health	<code>/health</code> returns 200 if DB reachable	Dev
Rollback	Simple <code>docker compose down</code> & up on Render (re-deploy)	Ops
Demo Reset	<code>scripts/reset_demo_data.py</code> drops & seeds DB (doctors + slots).	DevOps

16 Phase-Based Delivery Plan (8-Week Sprint)

Week	Milestone	Core Tasks	Deliverable
1	<i>Project bootstrap</i>	• Repo init, CI pipeline (lint + pytest). • Dockerfile, Render config. • PostgreSQL schema + seed script (doctors/slots). • Implement <code>/doctors</code>, <code>/available-slots</code>.	Running FastAPI with <code>/health</code> .
2	<i>Core booking API</i>	• State-machine logic (in-memory prototype). • <code>/book-appointment</code> with DB constraints.	Text-chat booking works via Postman.
3	<i>Conversation engine</i>	• Intent detection (regex + Gemini prompt). • <code>/chat</code> endpoint with state transition. • Unit tests for each transition.	End-to-end text booking flow (no voice).
4	<i>Speech-to-Text</i>	• Integrate Faster-Whisper (CPU-only). • <code>/transcribe</code> endpoint, store transcript rows. • Language detection fallback.	Speak → text appears in DB, passes to <code>/chat</code> .
5	<i>Text-to-Speech</i>	• XTTS model wrapper (async). • Return audio URL for streaming.	Voice responses audible in web UI.
6	<i>Dashboard</i>	• Streamlit app (metrics, tables, charts). • Auto-refresh via <code>@st.experimental_rerun</code>.	Live view of calls/bookings.
7	<i>Vapi phone integration</i> (optional but nice)	• Vapi webhook route (<code>/vapi-webhook</code>). • Receive inbound audio, pipeline through STT → chat → TTS → Vapi response.	Dial a number → get AI receptionist.

Week	Milestone	Core Tasks	Deliverable
8	<i>Polish & Demo</i>	• End-to-end demo script (see §17). • Reset script, README guide. • Architecture diagram (draw.io exported PNG).	Interview-ready repository – one-click start, <2-minute demo, 2-minute architecture walkthrough.

Each week ends with a living demo that can be run locally (`docker compose up`) and on Render (`render.com`).

17 Interview Demo Script (2-minute “show-and-tell”)

Step	Action	Expected Outcome
0	Open Streamlit dashboard (http://localhost:8501).	See 0 calls / 0 bookings.
1	In terminal, run <code>python demo_voice.py</code> (or click “Start Local Call” button). System prints “Listening...”.	Audio captured, sent to <code>/transcribe</code> .
2	Say: “Mujhe skin doctor ka appointment chahiye.”	STT returns Hindi text → <code>/chat</code> triggers <code>collect_specialization</code> .
3	System (XTTS) replies in Hindi: “Dr. Meera Sharma available kal 10 baje aur 4 baje. Kaun sa time chahiye?”	Dashboard now shows 1 call.
4	Say: “Kal 10 baje.”	System extracts slot, asks for name.
5	Say: “Mera naam Rahul Hai.”	System confirms: “Rahul, appointment with Dr. Meera Sharma tomorrow at 10 AM confirmed?”
6	Say: “Yes, confirm.”	Booking row created, slot marked booked.
7	Refresh dashboard (or auto-refresh). Shows 1 booking, 1 transcript. Click transcript → see Hindi user turn + English assistant turn.	Demonstrates full pipeline.
8	Explain architecture in < 2 min: audio → STT → state-machine → LLM → TTS; all data persisted → visible on dashboard.	Shows clear, interview-friendly story.

Feel free to repeat with a different language (e.g., “I need a dentist next Monday”) to showcase multilingual handling.

18 Resume One-Liner (copy-paste)

Built VaaniAI, a multilingual AI receptionist that books clinic appointments via voice (English, Hindi, Hinglish) using FastAPI, Gemini Flash, Faster-Whisper, XTTS, PostgreSQL, and Streamlit—delivered as a single-click demo that runs end-to-end in under two minutes.

19 Risks & Mitigations

Risk	Impact	Mitigation
STT accuracy for Hinglish	Mis-recognition may break intent flow.	Add simple keyword fallback (e.g., “doctor”, “appointment”, “kaun”). Provide a “I didn’t catch that” retry loop.
LLM hallucination (medical advice)	Could produce disallowed content.	Hard-code guardrails: any health-question not mapped to a doctor inquiry triggers safe-response (“I’m not a medical professional; I can help you book a doctor.”).
Slot double-booking	Overlap leads to bad demo.	DB unique constraint + transaction when booking; catch <code>IntegrityError</code> and ask for next slot.
API latency > 5 s	Demo feels sluggish.	Use local CPU-only models; keep audio clips ≤ 8 s; async FastAPI; pre-warm models on startup.
Vapi webhook timing out	Call drops.	Keep webhook response < 2 s; if TTS takes longer, return a “hold music” placeholder and stream later (out-of-scope for MVP).
Dashboard not refreshing	Data looks stale.	Use Streamlit’s <code>st.experimental_rerun</code> every 5 s or a “Refresh” button.
Missing environment variables on new machine	Setup fails.	Provide <code>.env.example</code> with clear instructions; <code>scripts/setup.sh</code> validates all keys.

20 Glossary (for interview talking points)

Term	Meaning
STT	Speech-to-Text – Faster-Whisper model.
LLM	Large Language Model – Gemini Flash (multilingual).
TTS	Text-to-Speech – XTTS (supports Hindi phonemes).
State-Machine	Deterministic flow controller; each user turn advances a known state (no agents).
Memory	Simple user profile stored in PostgreSQL (not vector store).
Dashboard	Streamlit UI for <i>ops visibility</i> – not a full CRM.
Vapi	Phone-call platform with webhook that forwards audio to our backend.
Render	Free-tier cloud host (Docker containers + managed PostgreSQL).

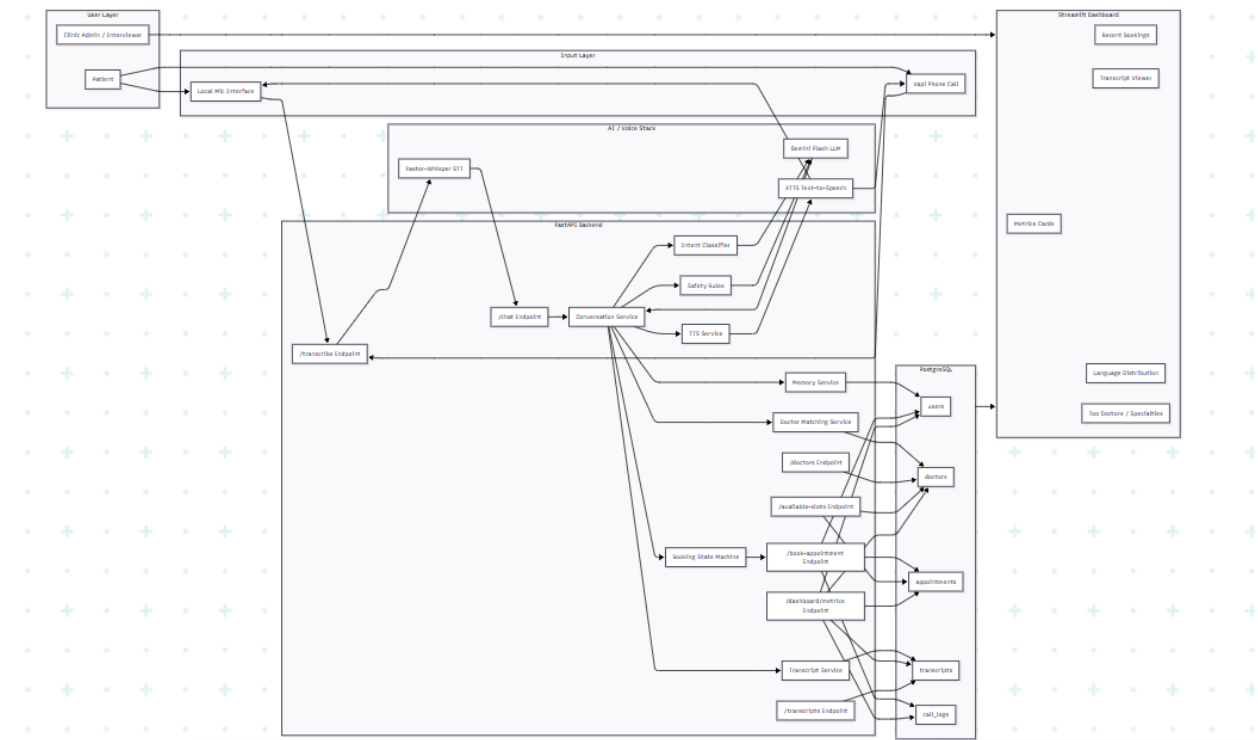
21 Final Review Checklist (Ready for Interview)

- ☒ Repo – `README.md` with “Run locally” and “Deploy to Render” sections.
 - ☒ Dockerfile – multi-stage (builder → runtime) ≤ 200 MB.
 - ☒ CI – GitHub Actions badge showing pass.
 - ☒ Tests – `pytest -q` passes, coverage $> 80\%$.
 - ☒ Seed data – `scripts/seed_doctors.py` creates 5 doctors and 2-3 slots each.
 - ☒ Demo script – `demo_voice.py` (or a simple `streamlit` UI with a “Speak” button).
 - ☒ Architecture diagram – PNG attached to repo.
 - ☒ One-click dashboard – `streamlit run dashboard.py`.
 - ☒ Resume line – ready to copy.
-

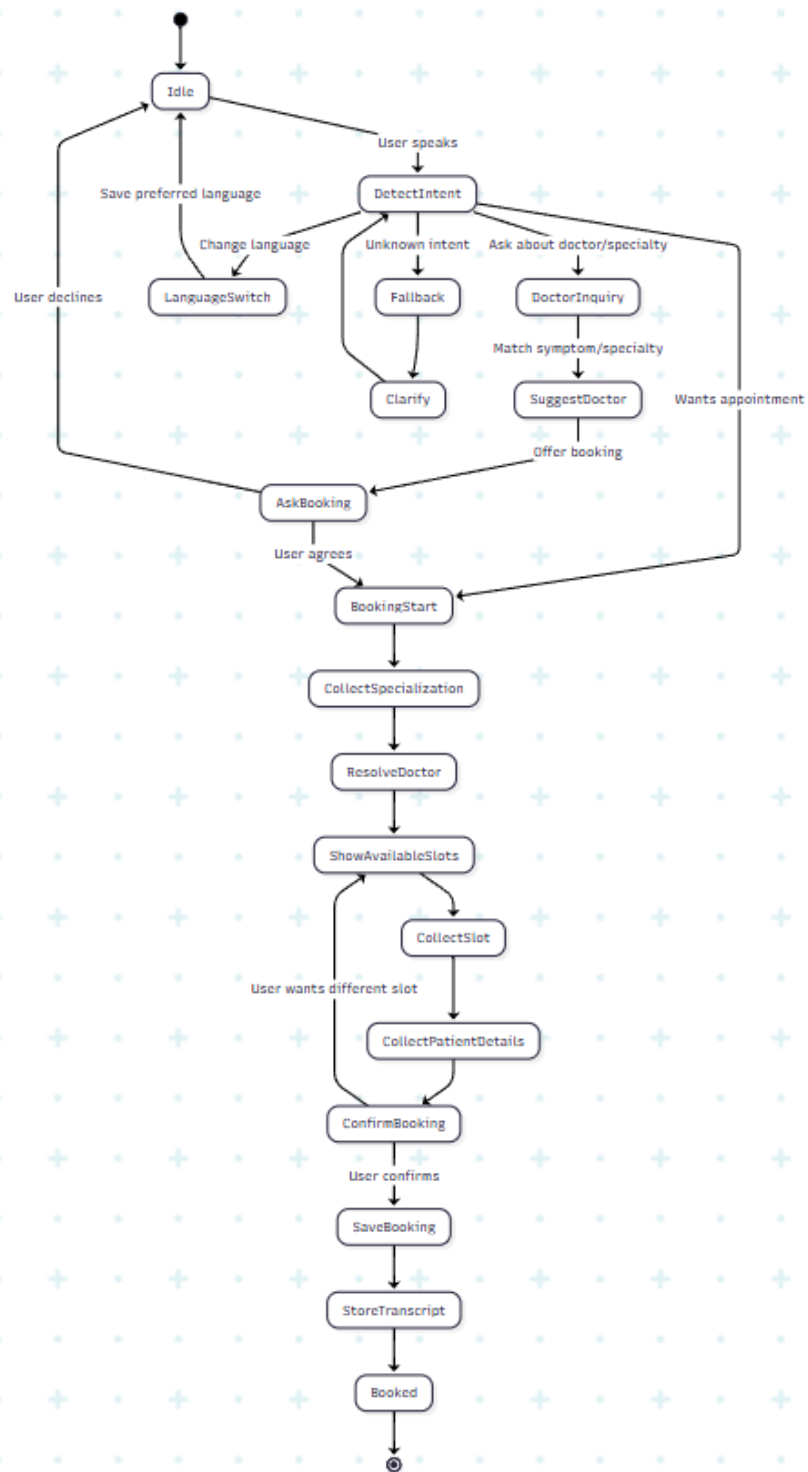
TL;DR – What you’ll say in the interview

“VaaniAI is a voice-first AI receptionist for a single clinic. The user says something in any of English, Hindi, or Hinglish; we feed the audio to Faster-Whisper, run a deterministic state machine that calls Gemini Flash for the next response, then synthesize the reply with XTTS. All conversation turns and the final appointment are stored in PostgreSQL, which powers a simple Streamlit dashboard. The whole stack lives in a single Docker container on Render, and the demo can be run end-to-end in under two minutes – perfect for interview showcase.”

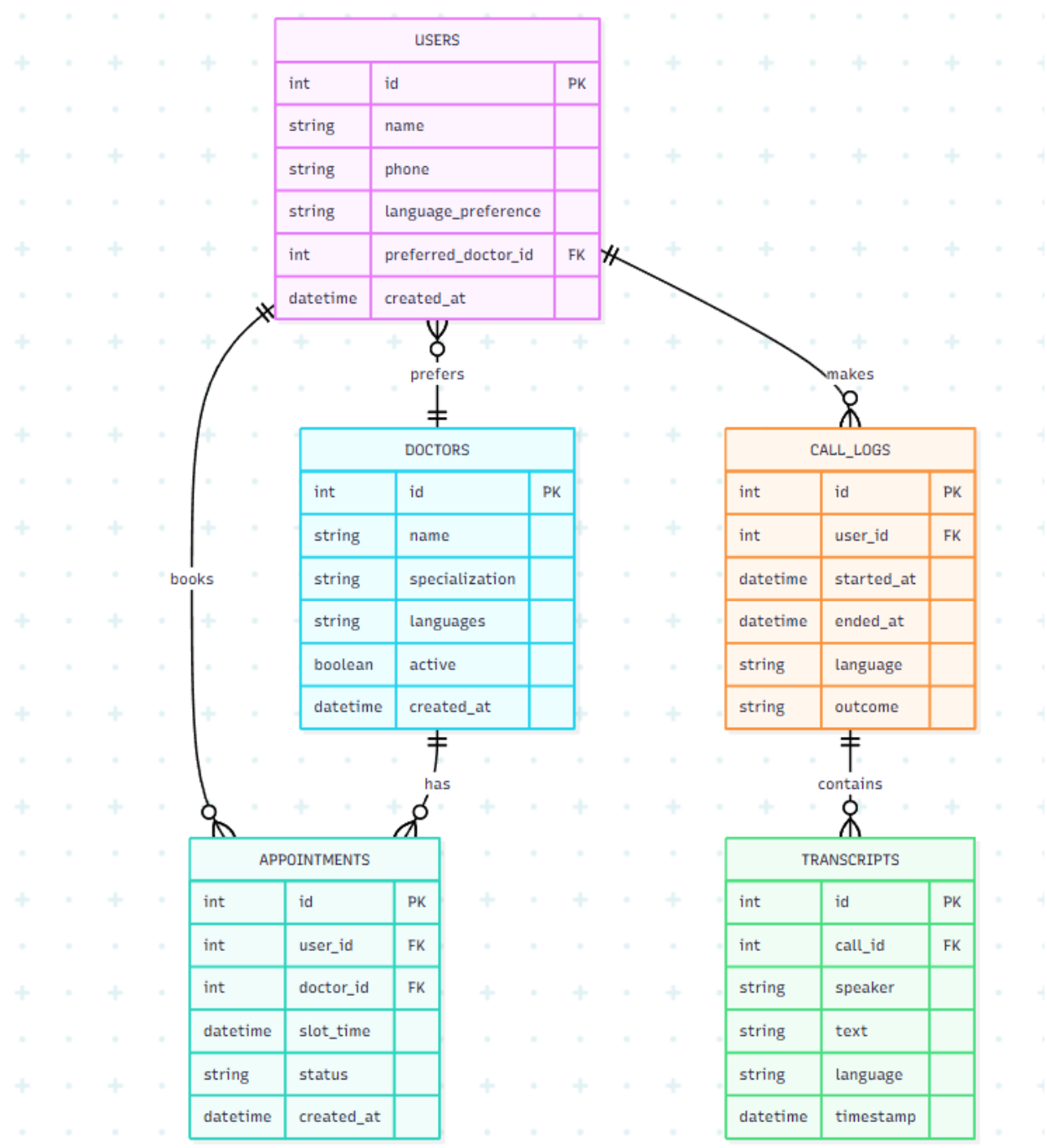
This PRD now contains everything you need to build, demo, and talk about VaaniAI in a technical interview. Good luck!



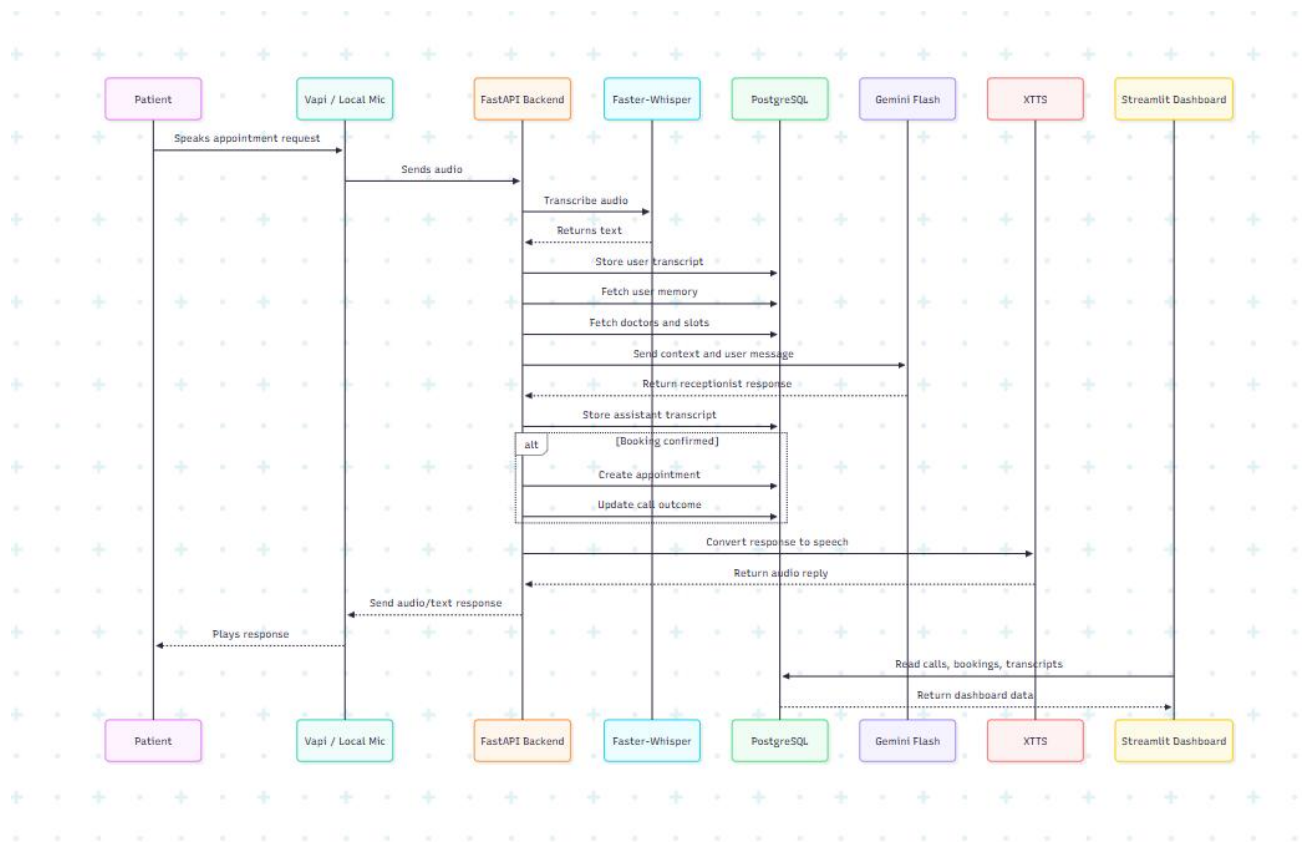
A more detailed version with backend modules and database tables:



Booking state machine



Database relationship diagram



End-to-end sequence diagram